

Drawing engine SVG parametrik

Panduan lengkap `schematic_test.js`

Koordinat engineering · Parameter ukuran · SVG pada halaman A4

Dokumen ini memuat seluruh penjelasan dari percakapan sebelumnya, termasuk 14 bagian, potongan kode, tabel, diagram teks, dan contoh perhitungan. Isi penjelasan dipertahankan; tata letaknya disesuaikan untuk PDF.

Catatan versi: contoh dalam penjelasan menggunakan parameter awal 6000 × 4000 mm dan format angka id-ID. File kerja yang diperiksa pada 18 September 2026 sudah menggunakan parameter 139.8 × 5000 mm, format en-US tanpa pemisah ribuan, serta beberapa label berbahasa Inggris. Penjelasan sebelumnya tetap disalin utuh sesuai permintaan.

Potongan kode menjelaskan bagian tertentu. Beberapa cuplikan sengaja hanya menampilkan sebagian fungsi dan bukan program mandiri.

Disusun: 18 September 2026

Isi dokumen

1. Menyiapkan parameter dan referensi HTML
2. `DrawingEngine`: kumpulan alat untuk menggambar
3. `toSvg()`: inti konversi koordinat engineering
4. `element()`: membuat elemen SVG
5. `paperLine()` dan `drawLine()`: dua tingkat koordinat
6. `drawPolyline()`: menggambar rangkaian garis
7. `drawText()`: menempatkan teks pada kertas
8. `drawArrow()`: membuat kepala panah dimensi
9. `drawDimension()`: menggambar satu dimensi lengkap
10. `drawRectangle()`: membentuk objek dari parameter
11. `render()`: menyusun seluruh halaman gambar
12. Menampilkan sumbu engineering
13. Mendengarkan perubahan input
14. Apa yang terjadi jika panjang diubah menjadi 8.000 mm?

JavaScript di `schematic_test.js` menghubungkan **parameter ukuran benda** → **perhitungan koordinat** → **elemen SVG**.

Ada dua jenis ukuran yang sengaja dipisahkan:

Jenis ukuran	Contoh	Satuan
Ukuran benda sebenarnya	Panjang 6.000, tinggi 4.000	mm engineering
Ukuran pada kertas	Jarak dimensi 14, ukuran panah 2,5	mm kertas

Pemisahan ini membuat benda dapat digambar dengan skala berbeda, sementara teks, panah, dan ketebalan garis tetap terbaca.

1. Menyiapkan parameter dan referensi HTML

JAVASCRIPT

```
"use strict";  
// Model: actual engineering dimensions in mm, independent of the screen.  
const parameters = { width: 6000, height: 4000 };
```

"use strict" mengaktifkan aturan JavaScript yang lebih ketat, misalnya mencegah penggunaan variabel yang belum dideklarasikan.

`parameters` menyimpan ukuran benda sebenarnya:

TEXT

```
Panjang = 6.000 mm  
Tinggi  = 4.000 mm
```

Objek ini menjadi sumber data untuk menggambar.

Walaupun dideklarasikan dengan `const`, isi propertinya tetap dapat diubah:

JAVASCRIPT

```
parameters.width = 8000; // Boleh
```

`const` mencegah variabel `parameters` diganti dengan objek lain.

Berikutnya:

JAVASCRIPT

```
const NS = "http://www.w3.org/2000/svg";  
const content = document.getElementById("drawing-content");  
const form = document.getElementById("parameters");  
const status = document.getElementById("status");
```

Fungsi masing-masing:

- `NS`: namespace untuk membuat elemen SVG.
- `content`: grup `<g>` tempat hasil gambar dimasukkan.
- `form`: form yang memuat input panjang dan tinggi.

- status: paragraf untuk menampilkan ukuran, skala, atau pesan kesalahan.

JAVASCRIPT

```
const format = (value) =>  
  Number(value.toFixed(2)).toLocaleString("id-ID");
```

Fungsi `format()` menyiapkan angka untuk ditampilkan.

Urutannya:

TEXT

```
6000.1234  
  ↓ toFixed(2)  
"6000.12"  
  ↓ Number(...)  
6000.12  
  ↓ toLocaleString("id-ID")  
"6.000,12"
```

Pembulatan ini digunakan untuk **teks tampilan**. Perhitungan geometri tetap memakai nilai aslinya.

2. DrawingEngine: kumpulan alat untuk menggambar

JAVASCRIPT

```
class DrawingEngine {  
  constructor(root, origin, scale) {  
    this.root = root;  
    this.origin = origin;  
    this.scale = scale;  
  }  
  // ...fungsi lainnya  
}
```

`class` mengelompokkan data dan fungsi yang saling berkaitan.

Setiap instance `DrawingEngine` menyimpan:

Properti	Isi
root	Grup SVG tujuan
origin	Posisi titik engineering (0,0) pada kertas
scale	Rasio ukuran kertas terhadap ukuran sebenarnya

Contohnya:

JAVASCRIPT

```
const engine = new DrawingEngine(  
  content,  
  { x: 40, y: 195 },  
  0.02  
);
```

Artinya:

- Hasil gambar masuk ke content.
- Titik engineering $(0,0)$ ditempatkan di posisi SVG $(40,195)$.
- Setiap 1 mm benda menjadi 0,02 mm pada kertas.
- Skalanya adalah 1:50.

this merujuk pada instance engine tersebut. Jadi `this.scale` adalah skala yang digunakan engine.

3. `toSvg()`: inti konversi koordinat engineering

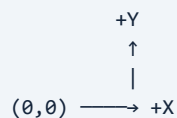
JAVASCRIPT

```
toSvg(xMm, yMm) {
  return {
    x: this.origin.x + xMm * this.scale,
    y: this.origin.y - yMm * this.scale,
  };
}
```

Ini bagian terpenting dari sistem koordinat.

Koordinat engineering:

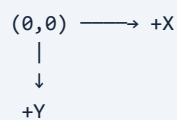
TEXT



$(0,0)$ ———→ +X
 ↑
 +Y

Koordinat SVG:

TEXT



$(0,0)$ ———→ +X
 ↓
 +Y

Karena arah Y berbeda, rumusnya menggunakan pengurangan:

JAVASCRIPT

```
y: this.origin.y - yMm * this.scale
```

Dengan contoh:

JAVASCRIPT

```
origin = { x: 40, y: 195 };
scale = 0.02;
```

Hasil konversinya:

Titik engineering	Titik SVG
$(0, 0)$	$(40, 195)$

Titik engineering	Titik SVG
(6000, 0)	(160, 195)
(6000, 4000)	(160, 115)
(0, 4000)	(40, 115)

Untuk sudut kanan atas:

TEXT

$x \text{ SVG} = 40 + 6000 \times 0,02 = 160$
 $y \text{ SVG} = 195 - 4000 \times 0,02 = 115$

Semua titik geometri benda dikonversi melalui fungsi ini. Karena itu, fungsi pembentuk benda tidak perlu menghitung arah koordinat layar sendiri.

4. `element()`: membuat elemen SVG

JAVASCRIPT

```

element(tag, attributes, text) {
  const node = document.createElementNS(NS, tag);
  for (const [key, value] of Object.entries(attributes)) {
    node.setAttribute(key, value);
  }
  if (text !== undefined) {
    node.textContent = text;
  }
  this.root.appendChild(node);
  return node;
}

```

Fungsi ini merupakan alat dasar untuk membuat elemen SVG apa pun.

Misalnya:

JAVASCRIPT

```

engine.element("line", {
  x1: 40,
  y1: 195,
  x2: 160,
  y2: 115,
  class: "object-line",
});

```

Hasilnya setara dengan:

HTML

```

<line
  x1="40"
  y1="195"
  x2="160"
  y2="115"
  class="object-line"
/>

```

Penjelasan urutannya:

1. `createElementNS(NS, tag)` membuat elemen dalam namespace SVG.
 2. `Object.entries(attributes)` mengambil pasangan nama dan nilai atribut.
 3. `setAttribute()` memasang setiap atribut.
 4. `textContent` mengisi teks jika diberikan.
 5. `appendChild()` memasukkan elemen ke grup gambar.
 6. `return node` mengembalikan elemen jika nantinya perlu dimodifikasi lagi.
- `createElementNS()` digunakan karena kita membuat elemen SVG melalui JavaScript.

5. `paperLine()` dan `drawLine()`: dua tingkat koordinat

JAVASCRIPT

```
paperLine(a, b, className = "dimension-line") {
  return this.element("line", {
    x1: a.x,
    y1: a.y,
    x2: b.x,
    y2: b.y,
    class: className,
  });
}
```

`paperLine()` langsung menerima koordinat **kertas/SVG**.

Contoh:

JAVASCRIPT

```
engine.paperLine(
  { x: 20, y: 30 },
  { x: 80, y: 30 }
);
```

Garis tersebut memiliki panjang 60 mm pada kertas.

Sementara itu:

JAVASCRIPT

```
drawLine(a, b, className = "object-line") {
  return this.paperLine(
    this.toSvg(a.x, a.y),
    this.toSvg(b.x, b.y),
    className
  );
}
```

`drawLine()` menerima koordinat **engineering**, lalu mengubahnya menjadi koordinat kertas.

Contoh:

JAVASCRIPT

```
engine.drawLine(
  { x: 0, y: 0 },
  { x: 6000, y: 0 }
);
```

Ini berarti menggambar garis benda sepanjang 6.000 mm. Panjangnya pada kertas mengikuti scale.

Alurnya:

```
TEXT
drawLine(titik engineering)
  ↓
toSvg()
  ↓
paperLine(titik kertas)
  ↓
element("line", ...)
```

Parameter:

```
JAVASCRIPT
className = "object-line"
```

adalah nilai default. Jika tidak diberikan class, garis menggunakan class object-line.

Class tersebut kemudian menentukan tampilannya melalui CSS.

6. drawPolyline(): menggambar rangkaian garis

```
JAVASCRIPT
drawPolyline(points, className = "object-line") {
  return this.element("polyline", {
    points: points.map(({ x, y }) => {
      const p = this.toSvg(x, y);
      return `${p.x},${p.y}`;
    }).join(" "),
    class: className,
  });
}
```

Fungsi ini menerima daftar titik engineering.

Misalnya:

```
JAVASCRIPT
[
  { x: 0, y: 0 },
  { x: 6000, y: 0 },
  { x: 6000, y: 4000 },
  { x: 0, y: 4000 },
  { x: 0, y: 0 },
]
```

Setiap titik dikonversi melalui toSvg().

Kemudian:

JAVASCRIPT

```
return `${p.x},${p.y}`;
```

mengubah satu titik menjadi teks seperti "40,195".

.join(" ") menggabungkan semuanya menjadi format yang dimengerti SVG:

HTML

```
<polyline points="40,195 160,195 160,115 40,115 40,195" />
```

Titik awal diulang pada akhir daftar supaya bentuknya tertutup.

Untuk gambar tiang nanti, prinsip yang sama dapat dipakai untuk membuat profil lurus atau meruncing.

7. drawText(): menempatkan teks pada kertas

JAVASCRIPT

```
drawText(  
  x,  
  y,  
  text,  
  className = "drawing-text",  
  attributes = {}  
) {  
  return this.element(  
    "text",  
    { x, y, class: className, ...attributes },  
    text  
  );  
}
```

Fungsi ini menggunakan koordinat **kertas**, bukan engineering.

Tujuannya agar judul, catatan, dan label dapat ditempatkan dengan ukuran serta jarak yang konsisten.

Contohnya:

JAVASCRIPT

```
engine.drawText(16, 20, "PERSEGI PANJANG PARAMETRIK");
```

Teks ditempatkan pada posisi $x = 16$, $y = 20$ di kertas.

`attributes = {}` memungkinkan pemberian atribut tambahan:

JAVASCRIPT

```
engine.drawText(100, 50, "6000 mm", "drawing-text", {  
  "text-anchor": "middle",  
});
```

`text-anchor: middle` membuat teks rata tengah terhadap posisi X.

Sintaks:

JAVASCRIPT

```
...attributes
```

memasukkan seluruh properti dari objek `attributes` ke dalam objek atribut SVG.

8. `drawArrow()`: membuat kepala panah dimensi

JAVASCRIPT

```
drawArrow(tip, direction) {  
  const length = 2.5;  
  const halfWidth = 0.65;  
  for (const side of [-1, 1]) {  
    this.paperLine(tip, {  
      x:  
        tip.x +  
        Math.cos(direction) * length -  
        Math.sin(direction) * halfWidth * side,  
      y:  
        tip.y +  
        Math.sin(direction) * length +  
        Math.cos(direction) * halfWidth * side,  
    });  
  }  
}
```

Panah dibuat dari dua garis pendek yang bertemu di `tip`.

Parameternya:

- `tip`: posisi ujung panah pada kertas.
- `direction`: arah dari ujung menuju bagian belakang kepala panah, dalam radian.
- `length`: panjang kepala panah pada kertas.
- `halfWidth`: setengah lebar kepala panah.

Perulangan:

JAVASCRIPT

```
for (const side of [-1, 1])
```

membuat dua sisi panah.

`Math.cos()` dan `Math.sin()` memutar posisi kedua sisi sesuai arah garis dimensi.

Karena fungsi ini menggunakan `paperLine()`, ukuran panah tetap sekitar 2,5 mm pada kertas walaupun ukuran benda berubah.

9. `drawDimension()`: menggambar satu dimensi lengkap

JAVASCRIPT

```
drawDimension(a, b, orientation, offset, label)
```

Fungsi ini membuat:

- Dua garis bantu.

- Satu garis dimensi.
- Dua kepala panah.
- Satu teks ukuran.

Contoh pemanggilan:

JAVASCRIPT

```
engine.drawDimension(
  { x: 0, y: 0 },
  { x: 6000, y: 0 },
  "horizontal",
  14,
  "6.000 mm"
);
```

Artinya: ukur dua titik engineering tersebut, lalu tempatkan garis dimensi horizontal 14 mm di bawahnya pada kertas.

Pertama, titik benda dikonversi:

JAVASCRIPT

```
const start = this.toSvg(a.x, a.y);
const end = this.toSvg(b.x, b.y);
const horizontal = orientation === "horizontal";
```

Kemudian posisi garis dimensi ditentukan:

JAVASCRIPT

```
const p = horizontal
  ? { x: start.x, y: start.y + offset }
  : { x: start.x + offset, y: start.y };
const q = horizontal
  ? { x: end.x, y: end.y + offset }
  : { x: end.x + offset, y: end.y };
```

Untuk dimensi horizontal, yang digeser adalah Y. Untuk dimensi vertikal, yang digeser adalah X.

Karena pergeseran ini dilakukan pada koordinat SVG:

Orientasi	Offset	Posisi
Horizontal	Positif	Bawah
Horizontal	Negatif	Atas
Vertikal	Positif	Kanan
Vertikal	Negatif	Kiri

Berikutnya:

JAVASCRIPT

```
const sign = Math.sign(offset);
```

sign bernilai 1 atau -1, untuk menentukan arah garis bantu.

Kode garis bantu memberi:

- Celah 1 mm dari objek.
- Tambahan panjang 2 mm melewati garis dimensi.

Setelah itu, garis utama dibuat:

JAVASCRIPT

```
this.paperLine(p, q);
```

Arah garis dihitung:

JAVASCRIPT

```
const angle = Math.atan2(q.y - p.y, q.x - p.x);
```

`Math.atan2()` menghasilkan sudut dari titik p menuju q.

Dua kepala panah dibuat dengan arah berlawanan:

JAVASCRIPT

```
this.drawArrow(p, angle);  
this.drawArrow(q, angle + Math.PI);
```

`Math.PI` radian sama dengan 180° .

Posisi teks dihitung dari titik tengah:

JAVASCRIPT

```
const x = (p.x + q.x) / 2 - (horizontal ? 0 : 2);  
const y = (p.y + q.y) / 2 - (horizontal ? 2 : 0);
```

Teks digeser 2 mm dari garis agar tidak bertumpuk dengan garis dimensi.

Untuk dimensi vertikal:

JAVASCRIPT

```
transform: `rotate(-90 ${x} ${y})`
```

memutar teks sebesar -90° pada titik penempatan teks.

Pada contoh ini, fungsi dimensi menangani dimensi horizontal dan vertikal dengan titik yang sejajar. Dimensi miring memerlukan pengembangan tambahan.

10. `drawRectangle()`: membentuk objek dari parameter

JAVASCRIPT

```
function drawRectangle(engine, model) {  
  const w = model.width;  
  const h = model.height;
```

`engine` menyediakan alat gambar. `model` menyediakan ukuran benda.

Bentuk persegi panjang ditentukan dengan lima titik:

JAVASCRIPT

```
engine.drawPolyline([
  { x: 0, y: 0 },
  { x: w, y: 0 },
  { x: w, y: h },
  { x: 0, y: h },
  { x: 0, y: 0 },
]);
```

Urutannya:

TEXT

A diagram of a rectangle. The bottom-left corner is labeled (0,0), the bottom-right corner is labeled (w,0), the top-right corner is labeled (w,h), and the top-left corner is labeled (0,h). The rectangle is drawn with solid lines, and the labels are placed outside the corners.

Tidak ada koordinat piksel di sini. Semua titik berasal dari ukuran benda.

Centerline dibuat di tengah objek:

JAVASCRIPT

```
const extension = 5 / engine.scale;
engine.drawLine(
  { x: w / 2, y: -extension },
  { x: w / 2, y: h + extension },
  "centerline"
);
engine.drawLine(
  { x: -extension, y: h / 2 },
  { x: w + extension, y: h / 2 },
  "centerline"
);
```

Centerline vertikal berada pada $x = w / 2$, sedangkan centerline horizontal berada pada $y = h / 2$.

Mengapa ada:

JAVASCRIPT

```
const extension = 5 / engine.scale;
```

Karena kita ingin centerline melewati objek sejauh **5 mm pada kertas**, sedangkan `drawLine()` menerima mm engineering.

Contoh skala 1:50:

TEXT

```
extension engineering = 5 / 0,02 = 250 mm
setelah digambar      = 250 × 0,02 = 5 mm kertas
```

Terakhir, dimensi ditambahkan:

JAVASCRIPT

```
engine.drawDimension(
```

```

    { x: 0, y: 0 },
    { x: w, y: 0 },
    "horizontal",
    14,
    `${format(w)} mm`
  );
  engine.drawDimension(
    { x: 0, y: 0 },
    { x: 0, y: h },
    "vertical",
    -14,
    `${format(h)} mm`
  );

```

Dimensi panjang berada di bawah objek. Dimensi tinggi berada di kiri objek.

Nilai teksnya diambil dari parameter, sehingga label dan geometri memakai sumber data yang sama.

11. render(): menyusun seluruh halaman gambar

JAVASCRIPT

```

function render(model) {
  content.replaceChildren();

```

replaceChildren() menghapus isi grup gambar sebelumnya. Dengan begitu, gambar baru tidak menumpuk di atas gambar lama.

Berikutnya, skala otomatis dihitung:

JAVASCRIPT

```

const scale = Math.min(
  140 / model.width,
  170 / model.height
);

```

Area untuk badan objek dibatasi menjadi 140 × 170 mm pada kertas.

Ada dua kandidat skala:

TEXT

```

Skala berdasarkan lebar = 140 / lebar benda
Skala berdasarkan tinggi = 170 / tinggi benda

```

Dipilih nilai terkecil agar kedua ukuran muat.

Untuk model 6.000 × 4.000 mm:

TEXT

```

140 / 6000 = 0,023333...
170 / 4000 = 0,0425
scale = 0,023333...

```

Ukuran objek pada kertas menjadi:

TEXT

```

Lebar = 6000 × 0,023333... = 140 mm

```

```
Tinggi = 4000 × 0,023333... = 93,33 mm
```

Skala yang ditampilkan adalah kebalikannya:

TEXT

```
1 / scale ≈ 42,86
```

Jadi sekitar **1:42,86**. Contoh ini memakai skala otomatis bebas, belum memilih skala gambar standar seperti 1:20 atau 1:50.

Posisi titik asal dihitung berikutnya:

JAVASCRIPT

```
const origin = {  
  x: 110 - model.width * scale / 2,  
  y: 148 + model.height * scale / 2,  
};
```

Tujuannya menempatkan **pusat objek pada titik kertas** (110,148).

- Posisi kiri = pusat X dikurangi setengah lebar.
- Posisi bawah = pusat Y ditambah setengah tinggi, karena Y SVG bertambah ke bawah.

Kemudian engine dibuat:

JAVASCRIPT

```
const engine = new DrawingEngine(content, origin, scale);
```

Judul dan catatan digambar memakai koordinat kertas:

JAVASCRIPT

```
engine.drawText(  
  16,  
  20,  
  "PERSEGI PANJANG PARAMETRIK",  
  "drawing-text heading"  
);
```

Objek utamanya digambar melalui:

JAVASCRIPT

```
drawRectangle(engine, model);
```

Pemisahan ini memungkinkan fungsi objek diganti atau ditambah nanti:

JAVASCRIPT

```
drawPole(engine, model);  
drawBasePlate(engine, model);
```

Engine dasarnya tetap dapat digunakan.

12. Menampilkan sumbu engineering

Di dalam `render()`:

JAVASCRIPT

```
const axisLength = 10 / scale;
```

Ini membuat panjang indikator sumbu tetap 10 mm pada kertas.

JAVASCRIPT

```
engine.drawLine(
  { x: 0, y: 0 },
  { x: axisLength, y: 0 },
  "axis-line"
);
engine.drawLine(
  { x: 0, y: 0 },
  { x: 0, y: axisLength },
  "axis-line"
);
```

Kedua garis dimulai dari titik engineering $(0,0)$.

Label $x \rightarrow$, $y \uparrow$, dan $(0,0)$ ditambahkan menggunakan koordinat kertas di sekitar origin.

Sumbu tersebut adalah penanda visual. Pembalikan arah Y yang sebenarnya tetap dilakukan oleh `toSvg()`.

Bagian akhir `render()` menampilkan ukuran dan skala pada kertas serta paragraf status HTML:

JAVASCRIPT

```
status.textContent =
  `Ukuran: ${format(model.width)} x ${format(model.height)} mm. ` +
  `Skala gambar  $\approx 1 : ${format(1 / scale)}.$ `;
```

13. Mendengarkan perubahan input

JAVASCRIPT

```
form.addEventListener("submit", (event) => event.preventDefault());
```

Ini mencegah form melakukan submit normal, misalnya ketika pengguna menekan Enter, sehingga halaman tidak dimuat ulang.

JAVASCRIPT

```
form.addEventListener("input", () => {
  if (!form.checkValidity()) {
    status.textContent =
      "Masukkan panjang dan tinggi antara 1 dan 100.000 mm. Gambar terakhir tetap ditampilkan.";
    return;
  }
  parameters.width = document.getElementById("width").valueAsNumber;
  parameters.height = document.getElementById("height").valueAsNumber;
  render(parameters);
});
```

Setiap input berubah:

1. `checkValidity()` memeriksa aturan HTML seperti `required`, `min`, dan `max`.
2. Jika tidak valid, status diperbarui dan fungsi berhenti.
3. Jika valid, nilai input dibaca sebagai angka melalui `valueAsNumber`.
4. Objek `parameters` diperbarui.
5. `render(parameters)` menggambar ulang.

Gambar terakhir tetap terlihat ketika input sementara kosong saat pengguna mengetik.

Baris terakhir:

JAVASCRIPT

```
render(parameters);
```

menggambar kondisi awal ketika halaman dibuka.

14. Apa yang terjadi jika panjang diubah menjadi 8.000 mm?

Alur lengkapnya:

TEXT

```
Input panjang: 8000
↓
Validasi input
↓
parameters.width = 8000
↓
render(parameters)
↓
Hitung skala dan origin baru
↓
drawRectangle(engine, parameters)
↓
Konversi titik engineering melalui toSvg()
↓
Buat elemen SVG baru
```

Dengan tinggi tetap 4.000 mm:

TEXT

```
scale = min(140 / 8000, 170 / 4000)
        = 0,0175
Lebar pada kertas = 140 mm
Tinggi pada kertas = 70 mm
```

Lebar pada kertas tetap 140 mm karena menggunakan skala otomatis, tetapi proporsinya berubah menjadi lebih mendatar dan label panjang berubah menjadi 8.000 mm.

Jika Anda ingin panjang pada kertas ikut bertambah ketika parameter panjang dinaikkan, gunakan skala tetap, misalnya:

JAVASCRIPT

```
const scale = 1 / 50;
```

Pada skala tersebut:

TEXT

Panjang 6.000 mm → 120 mm pada kertas

Panjang 8.000 mm → 160 mm pada kertas

Untuk pengembangan berikutnya, bagian yang paling sering Anda ubah adalah **model parameter** dan **fungsi pembentuk objek seperti** `drawRectangle()`. Fungsi konversi koordinat dan alat gambar dasar dapat terus digunakan untuk skematik yang lebih kompleks.